

CHAPITRE 7 : DÉVELOPPEMENT D'APPLICATIONS AUTOUR D'UNE BASE DE DONNÉES

Ce livret d'excellence haut de gamme regroupe :

- **Fiche 1 : Le Cours Complet** : Concepts fondamentaux de l'architecture d'application de base de données, l'interfaçage programmatique en Python, et la liaison visuelle dans l'environnement Lazarus.
- **Fiche 2 : Le Recueil de Questions-Réponses** : 40 Flashcards clés rédigées avec rigueur pour l'apprentissage actif.
- **Fiche 3 : Quiz d'Évaluation & Mémentos** : 10 questions types du Baccalauréat corrigées, tableaux de synthèse et mémo des erreurs classiques d'examen pratique.

Contents

1	Fiche 1 : Le Cours Complet	3
1.1	Introduction et Architecture d'Applications	3
1.2	Interfaçage Programmatique en Python (sqlite3 & mysql.connector)	3
1.2.1	Exemple Pratique : Gestion des Inscriptions en Python	4
1.3	Liaison Visuelle et Composants SQLdb dans Lazarus	5
1.3.1	Les Composants Clés de l'Architecture SQLdb	5
1.3.2	Schéma Conceptuel de Liaison Lazarus (TikZ)	5
1.3.3	Algorithme de Manipulation en Lazarus (Pascal)	6
2	Fiche 2 : Le Recueil de Questions-Réponses (40 Flashcards)	7
3	Fiche 3 : Quiz d'Évaluation & Fiches Mémo	12
3.1	Quiz d'Évaluation (10 Questions Types Bac)	12
3.2	Mémento de l'Étudiant : Tableau de Synthèse	14
3.3	Les 5 Pièges Classiques à Éviter le Jour de l'Épreuve Pratique	14

Chapter 1

Fiche 1 : Le Cours Complet

1.1 Introduction et Architecture d'Applications

Dans les chapitres précédents, nous avons appris à structurer, créer, modifier et manipuler une base de données à l'aide de requêtes SQL. Cependant, l'utilisateur final d'un système informatique n'utilise pas de console de requêtes brutes SQL. Il interagit avec une **interface graphique utilisateur** (GUI) contenant des formulaires, des boutons et des grilles visuelles.

Le développement d'applications autour d'une base de données consiste à lier un langage de programmation (comme **Python** ou **Object Pascal/Lazarus**) à un Système de Gestion de Bases de Données (SGBD).

Architecture d'une Application Connectée

Une architecture standard se décompose en trois composants physiques ou logiques :

1. **L'Interface Utilisateur (Présentation)** : Fenêtres, zones de saisie, boutons (ex. formulaires Lazarus, widgets PyQt/Tkinter en Python).
2. **Le Noyau Applicatif (Logique métier)** : Code source assurant le contrôle des règles de gestion, la construction dynamique des requêtes SQL et la réception des résultats.
3. **Le Serveur de Données (Stockage)** : Le SGBD (ex. MySQL, PostgreSQL, SQLite, MS Access) qui exécute physiquement les requêtes, assure la cohérence des données et gère les fichiers de la base.

1.2 Interfaçage Programmatique en Python (sqlite3 & mysql.connector)

Le langage Python fournit des bibliothèques standards ou tierces permettant de manipuler des bases de données. Pour des raisons d'apprentissage et de portabilité au Baccalauréat, deux connecteurs sont principalement étudiés : `sqlite3` (base de données locale embarquée) et `mysql.connector` (base de données serveur distribuée).

Le Cycle de Vie de la Connexion en Python

Toute interaction avec une base de données en Python suit rigoureusement un protocole en 6 étapes :

1. **Importation du connecteur** : `import sqlite3` ou `import mysql.connector`.
2. **Établissement de la Connexion** : Création d'un objet de connexion reliant le script au fichier ou serveur de données.
3. **Création du Curseur** : L'outil logique permettant de formuler les requêtes SQL et de naviguer parmi les résultats renvoyés par la base.
4. **Exécution des instructions SQL** : Transmission des requêtes LDD ou LMD.
5. **Validation (Commit)** : Obligatoire pour enregistrer définitivement toute modification d'écriture (INSERT, UPDATE, DELETE).

6. **Fermeture des objets** : Restitution des ressources système en fermant le curseur puis l'objet de connexion.

1.2.1 Exemple Pratique : Gestion des Inscriptions en Python

Le code ci-dessous illustre une implémentation d'insertion sécurisée de données au sein d'une base SQLite3 :

Code Python d'Insertion Sécurisée

```
import sqlite3

def inscrire_eleve(num_inscription, nom, prenom, code_classe):
    try:
        # 1. Connexion au fichier de la base de données
        conn = sqlite3.connect("scolaire.db")
        cur = conn.cursor()

        # 2. Requête paramétrée pour éviter les injections SQL
        requete = "INSERT INTO ELEVE (num, nom, prenom, id_classe) VALUES (?, ?, ?, ?)"
        donnees = (num_inscription, nom, prenom, code_classe)

        # 3. Exécution de la requête
        cur.execute(requete, donnees)

        # 4. Validation impérative de la transaction
        conn.commit()
        print("Élève inscrit avec succès d'excellence !")

    except sqlite3.Error as e:
        print(f"Erreur de base de données rencontrée : {e}")
        # En cas d'erreur, on annule la transaction courante
        conn.rollback()

    finally:
        # 5. Fermeture propre des descripteurs
        if cur:
            cur.close()
        if conn:
            conn.close()
```

1.3 Liaison Visuelle et Composants SQLdb dans Lazarus

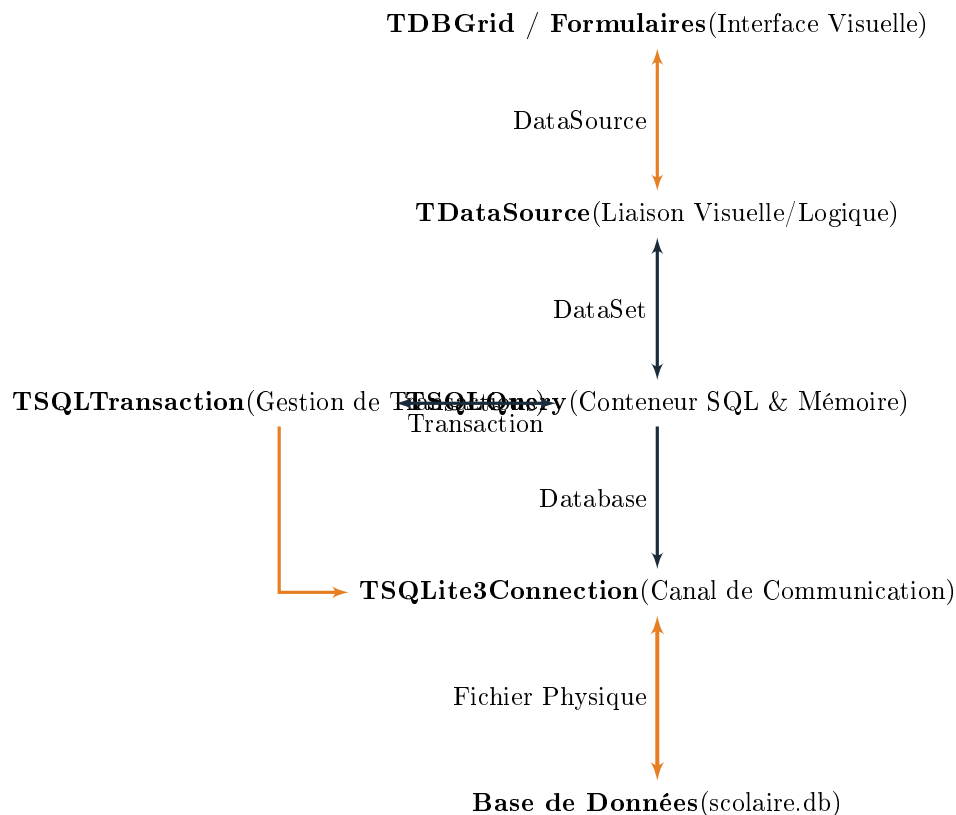
L'environnement Lazarus propose une approche visuelle appelée **SQLdb** pour lier une base de données relationnelle à une interface utilisateur. Cette architecture repose sur un empilement logique de composants indispensables qui forment une chaîne de transmission ininterrompue.

1.3.1 Les Composants Clés de l'Architecture SQLdb

- **Le Connecteur Physique** (TMySQL57Connection ou TSQLite3Connection) : Gère la communication physique avec le fichier local ou le serveur distant de la base (Adresse IP, port, nom de la base, utilisateur et mot de passe).
- **Le Gestionnaire de Transaction** (TSQLTransaction) : Assure le contrôle ACID des opérations logiques d'écriture. Il valide définitivement (**Commit**) ou annule complètement (**Rollback**) les requêtes exécutées.
- **La Requête SQL** (TSQLQuery) : Contient la chaîne d'instruction SQL brute via sa propriété `SQL.Text`. Elle récupère les résultats ou applique les changements.
- **La Liaison Logique** (TDataSource) : Fait office de passerelle sémantique entre le composant non visuel TSQLQuery et les composants visuels de l'interface graphique.
- **La Grille Visuelle** (TDBGrid) : Contrôle graphique interactif qui affiche sous forme de tableau bidimensionnel les lignes et colonnes issues d'un TSQLQuery via l'intermédiaire du TDataSource.

1.3.2 Schéma Conceptuel de Liaison Lazarus (TikZ)

Le schéma vectoriel ci-dessous illustre visuellement le flux d'informations et les dépendances logiques liant l'interface utilisateur de l'application cliente à la base de données physique :



1.3.3 Algorithme de Manipulation en Lazarus (Pascal)

Dans Lazarus, pour afficher les résultats d'une requête dynamique saisie par un utilisateur, on utilise la méthode d'activation de requêtes de consultation.

Code Pascal de Consultation de Données

Le code suivant permet de rafraîchir un composant de requête visuel à l'aide d'un filtre personnalisé saisi au clavier :

```
procedure TForm1.BtnRechercherClick(Sender: TObject);
begin
    // 1. Désactivation temporaire de la requête active
    SQLQuery1.Active := False;

    // 2. Nettoyage de la commande SQL précédente
    SQLQuery1.SQL.Clear;

    // 3. Affectation du nouveau texte SQL avec paramètre
    SQLQuery1.SQL.Text := 'SELECT * FROM ELEVE WHERE nom LIKE :ParamNom';

    // 4. Attribution de la valeur saisie dans un champ d'édition (Edit1)
    SQLQuery1.ParamByName('ParamNom').AsString := Edit1.Text + '%';

    // 5. Réactivation et chargement des données dans la grille visuelle
    SQLQuery1.Active := True;
end;
```

Code Pascal d'Exécution de Requête d'Écriture (LMD)

Pour les instructions qui modifient la structure ou les données de la base (INSERT, UPDATE, DELETE), on n'utilise pas la propriété `Active := True` mais la méthode `ExecSQL` combinée à la gestion transactionnelle :

```
procedure TForm1.BtnInscrireClick(Sender: TObject);
begin
    try
        SQLQuery1.Active := False;
        SQLQuery1.SQL.Clear;
        SQLQuery1.SQL.Add('INSERT INTO ELEVE (num, nom, prenom) VALUES (:Num, :Nom, :Pren)');
        SQLQuery1.ParamByName('Num').AsInteger := StrToInt(EditNum.Text);
        SQLQuery1.ParamByName('Nom').AsString := EditNom.Text;
        SQLQuery1.ParamByName('Pren').AsString := EditPrenom.Text;

        // Étape cruciale : exécution physique de l'écriture
        SQLQuery1.ExecSQL;

        // Validation définitive de la transaction active
        SQLTransaction1.Commit;
        ShowMessage('Élève enregistré et validé !');
    except
        on E: Exception do
            begin
                SQLTransaction1.Rollback;
                ShowMessage('Erreur : ' + E.Message + '. Transaction annulée.');
            end;
        end;
    end;
```

Chapter 2

Fiche 2 : Le Recueil de Questions-Réponses (40 Flashcards)

Flashcard 1 : Quel est le but ultime du développement d'applications autour d'une base de données ?

Réponse : Permettre à des utilisateurs non techniciens d'interagir de façon sécurisée, visuelle et intuitive avec des données persistantes via une interface utilisateur (GUI) sans avoir à saisir manuellement des lignes de requêtes SQL brutes.

Flashcard 2 : Qu'est-ce qu'une interface graphique utilisateur (GUI) ?

Réponse : C'est l'ensemble visuel composé de formulaires, de boutons, de listes déroulantes et de grilles de données avec lesquels l'utilisateur final interagit directement pour saisir et consulter des informations.

Flashcard 3 : Qu'entend-on par architecture client-serveur de base de données ?

Réponse : C'est une structure où une application cliente (située sur la machine utilisateur) formule et transmet des requêtes de données à un serveur distant qui héberge le SGBD, exécute les calculs et lui renvoie uniquement les résultats.

Flashcard 4 : Quel est le rôle spécifique du connecteur de base de données en Python ?

Réponse : C'est une bibliothèque (ex. `sqlite3` ou `mysql.connector`) servant d'interface intermédiaire pour traduire les instructions Python en commandes interprétables par le SGBD et récupérer les flux de données.

Flashcard 5 : Quelle est la différence fondamentale entre SQLite3 et MySQL ?

Réponse : SQLite3 est un SGBD autonome embarqué sans serveur, où l'intégralité de la base est stockée dans un unique fichier local. MySQL est un SGBD relationnel client-serveur nécessitant un service d'arrière-plan actif et gérant des accès concurrents distants.

Flashcard 6 : Pourquoi doit-on toujours fermer la connexion à une base de données en fin de script ?

Réponse : Pour libérer instantanément les descripteurs de fichiers système, restituer la mémoire vive allouée en RAM et fermer proprement les sockets réseau ouverts pour éviter les fuites de ressources.

Flashcard 7 : À quoi sert l'objet Curseur (Cursor) en programmation de bases de données ?

Réponse : C'est un pointeur logique mémoire qui permet d'exécuter des instructions SQL et de naviguer pas-à-pas (à l'aide de méthodes de récupération comme `fetchall()`) à travers l'ensemble des enregistrements renvoyés.

Flashcard 8 : Que fait précisément la méthode `commit()` de la connexion Python ?

Réponse : Elle applique de manière définitive et permanente toutes les modifications d'écriture effectuées lors de la transaction courante sur le support physique de stockage de la base.

Flashcard 9 : Que se passe-t-il si un script Python modifie des lignes de données sans exécuter de `commit()` ?

Réponse : Toutes les modifications sont stockées temporairement dans un tampon mémoire virtuel et sont entièrement annulées et perdues dès la fermeture de la connexion ou l'arrêt du programme.

Flashcard 10 : Quel est le but de l'instruction `rollback()` en cas d'exception ?

Réponse : Annuler instantanément l'intégralité des requêtes d'écriture exécutées depuis le dernier `commit` afin de ramener la base de données à un état antérieur cohérent et stable.

Flashcard 11 : Qu'est-ce qu'une requête paramétrée et quel est son principal atout technique ?

Réponse : C'est une requête contenant des jetons d'emplacement (`?` en SQLite, `%s` en MySQL) complétés par des variables indépendantes. Son principal atout est de bloquer définitivement les failles de sécurité de type **Injection SQL**.

Flashcard 12 : Comment l'injection SQL altère-t-elle le comportement d'une application ?

Réponse : En injectant des instructions SQL de manière frauduleuse dans les champs de saisie de texte utilisateurs, détournant la logique interne du programme pour voler des informations confidentielles ou détruire des tables.

Flashcard 13 : Quel est le rôle de la méthode `fetchall()` d'un curseur en Python ?

Réponse : Elle extrait et retourne la totalité des enregistrements restants renvoyés par une requête de sélection sous la forme d'une liste de tuples Python.

Flashcard 14 : En quoi la méthode `fetchone()` se distingue-t-elle de `fetchall()` ?

Réponse : `fetchone()` extrait uniquement la ligne suivante du jeu de résultats sous la forme d'un seul tuple, renvoyant `None` si aucune ligne n'est disponible, optimisant ainsi la mémoire RAM.

Flashcard 15 : Quel est l'équivalent Lazarus d'un curseur mémoire et d'un conteneur SQL ?

Réponse : Le composant non visuel `TSQLQuery`. Il détient la propriété textuelle de l'instruction (`SQL.Text`) et encapsule les enregistrements chargés en mémoire vive.

Flashcard 16 : À quoi correspond l'extension SQLdb dans l'environnement Lazarus ?

Réponse : C'est l'onglet regroupant l'intégralité des composants non visuels spécialisés de Lazarus permettant de configurer la liaison, la manipulation et les transactions sur bases de données.

Flashcard 17 : Quel est le rôle du composant TSQLite3Connection dans Lazarus ?

Réponse : Établir la connexion logique et physique entre l'application et le fichier de base de données SQLite3, en renseignant la propriété `DatabaseName` avec le chemin du fichier.

Flashcard 18 : Pourquoi le composant TSQLTransaction est-il techniquement indispensable dans Lazarus ?

Réponse : Parce que les composants de requêtes SQLdb ne peuvent pas s'activer sans être associés à une transaction qui supervise la gestion de la cohérence et de la persistance de l'échange.

Flashcard 19 : Comment associe-t-on logiquement une transaction à un connecteur dans Lazarus ?

Réponse : En paramétrant la propriété `Transaction` de l'objet de connexion sur le composant `TSQLTransaction` instancié, et réciproquement en affectant la propriété `Database` de la transaction au connecteur.

Flashcard 20 : Quel est le rôle de la propriété SQL d'un composant TSQLQuery ?

Réponse : C'est une liste de chaînes de caractères (de type `TStrings`) qui contient le code source de l'instruction SQL à transmettre et exécuter sur la base de données.

Flashcard 21 : Quelle est la différence entre les méthodes `SQLQuery1.Open` et `SQLQuery1.ExecSQL` ?

Réponse : `Open` (ou `Active := True`) est exclusivement réservée aux requêtes de consultation (`SELECT`) qui renvoient un jeu de résultats. `ExecSQL` est requise pour les requêtes de modification (`INSERT`, `UPDATE`, `DELETE`) sans retour de lignes.

Flashcard 22 : Qu'est-ce que le composant TDataSource et quel est son rôle ?

Réponse : C'est un adaptateur non visuel qui fait office de pont logique entre le composant d'accès aux données (`TSQLQuery`) et les composants d'affichage graphique de l'interface utilisateur.

Flashcard 23 : Comment lie-t-on visuellement une grille TDBGrid à une requête TSQLQuery ?

Réponse : On associe la propriété `DataSet` du `TDataSource` au composant `TSQLQuery`, puis la propriété `DataSource` de la grille `TDBGrid` au `TDataSource`.

Flashcard 24 : À quel moment le composant TDBGrid affiche-t-il les lignes de la base de données ?

Réponse : Dès que la connexion, la transaction, le datasource et la grille sont connectés, et que la propriété `Active` du composant `TSQLQuery` passe à la valeur `True`.

Flashcard 25 : Quel est l'intérêt d'appeler `SQLQuery1.Close` (ou `Active := False`) avant de modifier sa propriété `SQL.Text` ?

Réponse : Réinitialiser l'état du composant mémoire, vider le jeu de résultats précédent et désallouer les ressources de l'ancienne requête pour pouvoir en instancier une nouvelle en toute sécurité.

Flashcard 26 : Comment définit-on un paramètre dynamique dans la propriété `SQL` d'un `TSQLQuery` ?

Réponse : En préfixant l'identifiant du paramètre par le symbole double-point (`:`) dans le corps de la requête textuelle (Exemple : `WHERE age > :MinAge`).

Flashcard 27 : Comment attribue-t-on une valeur à un paramètre nommé dans Lazarus ?

Réponse : En utilisant la méthode d'affectation typée : `SQLQuery1.ParamByName('NomDuParametre').AsType := Valeur` (où `AsType` correspond à `AsString`, `AsInteger`, etc.).

Flashcard 28 : Quel est le comportement de l'instruction `SQLQuery1.SQL.Add('...')` par rapport à `SQLQuery1.SQL.Text := '...'` ?

Réponse : `Add` ajoute une nouvelle ligne à la suite de la liste existante d'instructions textuelles, tandis que l'affectation à `Text` remplace instantanément l'intégralité du contenu par la nouvelle chaîne.

Flashcard 29 : Quelle méthode de `TSQLTransaction` valide définitivement les écritures de Lazarus en maintenant la structure ouverte ?

Réponse : C'est la méthode `CommitRetaining` de l'objet transaction. Elle applique définitivement les modifications sur la base de données tout en maintenant les connexions et les composants actifs.

Flashcard 30 : Que réalise la procédure `RollbackRetaining` ?

Réponse : Elle annule l'intégralité des actions de la transaction en cours tout en évitant la désactivation et la réouverture fastidieuse de l'ensemble des connexions et formulaires.

Flashcard 31 : Quel composant visuel de Lazarus permet de modifier ou insérer des enregistrements sans écrire de lignes de code ?

Réponse : Le composant `TDBNavigator`. Il fournit à l'utilisateur des boutons de contrôle physiques de type "Suivant", "Ajouter", "Supprimer", "Éditer" et "Valider" directement liés au `TDataSource`.

Flashcard 32 : Comment gère-t-on le mot de passe et l'identifiant de connexion MySQL de manière sécurisée en Lazarus ?

Réponse : En paramétrant les propriétés `UserName` et `Password` du connecteur physique au cours de l'initialisation du programme, ou dynamiquement via une boîte de dialogue de connexion.

Flashcard 33 : Qu'est-ce que le fichier de bibliothèque client (ex. `sqlite3.dll` ou `libmysql.dll`) et pourquoi est-il requis par Lazarus ?

Réponse : C'est le pilote compilé natif fourni par le SGBD. L'exécutable généré par Lazarus doit pouvoir charger cette bibliothèque dynamique pour pouvoir émettre physiquement les sockets réseau vers la base.

Flashcard 34 : Comment l'architecture Merise ou le dictionnaire de données influencent-ils le développement d'applications ?

Réponse : Ils garantissent que l'application respecte les types d'attributs déclarés, les longueurs maximales de chaînes et les règles comportementales de clés étrangères lors de la conception des formulaires de saisie.

Flashcard 35 : Que se passe-t-il au niveau applicatif si l'utilisateur saisit une valeur violant une contrainte d'intégrité de domaine ?

Réponse : Le SGBD rejette l'instruction SQL lors de son exécution, renvoie un code d'erreur applicatif que l'application doit intercepter (via un bloc `try..except` ou `try..catch`) pour informer proprement l'utilisateur.

Flashcard 36 : Pourquoi ne doit-on jamais utiliser la fonction `StrToInt` sur un champ de texte de formulaire sans vérification préalable ?

Réponse : Parce que si l'utilisateur saisit accidentellement un caractère alphabétique, `StrToInt` provoquera un plantage immédiat par exception de conversion avant même que la requête ne soit transmise.

Flashcard 37 : Quel est le but d'un bloc de gestion des exceptions `try..except..end` en Lazarus Pascal ?

Réponse : Sécuriser l'exécution des opérations critiques sur la base de données afin d'éviter le plantage physique de l'exécutable en cas d'erreur de saisie, de connexion réseau ou de contrainte.

Flashcard 38 : Quel composant de Lazarus permet de lier des zones de texte indépendantes à des attributs spécifiques d'une base ?

Réponse : Le composant `TDBEdit`. Contrairement au `TEdit` standard, il se connecte directement à un attribut de la table active via les propriétés `DataSource` et `DataField`.

Flashcard 39 : Comment simule-t-on le résultat d'une jointure SQL complexe au sein d'une grille `TDBGrid` ?

Réponse : En intégrant directement une instruction SQL de sélection de jointure multi-tables dans le corps de la propriété `SQL.Text` du composant `TSQLQuery` lié (ex. `SELECT * FROM CLIENT C JOIN FACTURE F ON C.id = F.id_client`).

Flashcard 40 : Quelle est l'importance de l'indépendance logique des données pour le développement d'applications clientes ?

Réponse : Elle permet d'ajouter ou modifier la structure logique interne de la base de données (ex. ajouter de nouveaux attributs) sans perturber ni exiger de réécriture intégrale du code source de l'interface cliente.

Chapter 3

Fiche 3 : Quiz d'Évaluation & Fiches Mémo

3.1 Quiz d'Évaluation (10 Questions Types Bac)

Question 1 : Quelle méthode est obligatoire pour exécuter une requête d'insertion en Python ?

- A) `cur.fetchall()`
- B) `cur.execute()` et `conn.commit()`
- C) `cur.open()`

Correction détaillée : La bonne réponse est la **B**. L'exécution de la requête via `execute()` prépare le changement en mémoire tampon, mais seul l'appel de la méthode de validation `commit()` de l'objet de connexion écrit physiquement les données de manière permanente.

Question 2 : VRAI ou FAUX : En Lazarus, l'opération `SQLQuery1.Active := True` s'applique sur les requêtes `DELETE`.

Correction détaillée : **FAUX**. L'activation de la propriété `Active` équivaut à la méthode `Open`, réservée aux requêtes renvoyant un jeu d'enregistrements (`SELECT`). Pour les requêtes d'écriture et de suppression (`DELETE`), il faut utiliser l'instruction `ExecSQL`.

Question 3 : Quel composant Lazarus sert d'intermédiaire direct de transmission entre la base de données et l'interface utilisateur ?

- A) `TDataSource`
- B) `TSQLTransaction`
- C) `TButton`

Correction détaillée : La bonne réponse est la **A**. Le composant `TDataSource` fait l'interface sémantique en se liant d'un côté au jeu de données (`TSQLQuery`) et de l'autre aux contrôles d'affichage graphique de la fenêtre utilisateur.

Question 4 : VRAI ou FAUX : Une requête paramétrée protège l'application contre les attaques par injection SQL.

Correction détaillée : **VRAI**. En séparant le code de l'instruction des données de paramètres dynamiques, le moteur du SGBD traite les paramètres uniquement comme des valeurs littérales de données et jamais comme du code exécutable.

Question 5 : Quelle méthode Python permet de vider l'ensemble d'une transaction non validée en cas d'erreur ?

- A) `conn.rollback()`
- B) `conn.close()`
- C) `cur.clear()`

Correction détaillée : La bonne réponse est la **A**. La méthode `rollback()` annule immédiatement toutes les requêtes LMD de la transaction courante non validées, restaurant ainsi la cohérence d'origine de la base.

Question 6 : VRAI ou FAUX : Le composant TDBGrid peut modifier directement les données si sa requête associée n'est pas active.

Correction détaillée : **FAUX**. La grille visuelle n'est qu'un afficheur interactif. Si la requête `TSQLQuery` sous-jacente est désactivée (`Active = False`), aucun enregistrement n'est chargé en mémoire, rendant la grille inactive et vide de tout contenu.

Question 7 : En Lazarus, comment déclare-t-on un paramètre nommé dans le corps du texte SQL ?

- A) En l'entourant de guillemets simples
- B) En le préfixant d'un point d'interrogation ?
- C) En le préfixant de deux-points :

Correction détaillée : La bonne réponse est la **C**. Dans la syntaxe Lazarus Pascal, les paramètres dynamiques sont déclarés par l'usage des deux-points (ex. : `MonParametre`) et se valorisent à l'aide de la méthode `ParamByName`.

Question 8 : VRAI ou FAUX : La méthode `fetchone()` de Python retourne une liste contenant tous les tuples de la requête.

Correction détaillée : **FAUX**. La fonction `fetchone()` retourne uniquement un unique tuple contenant l'enregistrement physique immédiatement suivant du pointeur. Pour extraire la liste complète, on doit appeler la méthode `fetchall()`.

Question 9 : Quel composant Lazarus configure la communication réseau et physique avec un serveur de base de données MySQL ?

- A) `TMySQL57Connection`
- B) `TSQLQuery`
- C) `TDBGrid`

Correction détaillée : La bonne réponse est la **A**. C'est le rôle exclusif du connecteur physique (ici `TMySQL57Connection`) de définir l'adresse réseau IP de destination (`HostName`), l'utilisateur et le mot de passe d'accès.

Question 10 : VRAI ou FAUX : En cas de violation de contrainte d'intégrité référentielle, le programme Python s'arrête brutalement sauf s'il est protégé par un bloc `try-except`.

Correction détaillée : **VRAI**. L'erreur renvoyée par le SGBD se traduit en Python par la levée d'une exception de type `DatabaseError`. Si elle n'est pas encapsulée et gérée, le programme principal est immédiatement interrompu par le système d'exploitation.

3.2 Mémento de l'Étudiant : Tableau de Synthèse

Table 3.1: Synthèse Technique des Équivalences Conceptuelles de Développement

Objectif Conceptuel	Implémentation Python	Composant Lazarus SQLdb
Connexion au SGBD	<code>sqlite3.connect()</code> ou <code>mysql.connector.connect()</code>	<code>TSQLite3Connection</code> ou <code>TMySQL57Connection</code>
Encapsulation de Transaction	Implicite (géré par l'objet <code>connection</code>)	<code>TSQLTransaction</code>
Conteneur et exécuteur SQL	Objet <code>Cursor</code> via <code>cur.execute()</code>	<code>TSQLQuery</code>
Liaison de données	Variable liste de tuples en mémoire vive	<code>TDataSource</code>
Affichage interactif GUI	Widget Tkinter (<code>Ttreeview</code>) ou PyQt (<code>QTableWidget</code>)	<code>TDBGrid</code>
Annulation de Transaction	<code>conn.rollback()</code>	<code>SQLTransaction1.Rollback</code>
Validation de Transaction	<code>conn.commit()</code>	<code>SQLTransaction1.Commit</code>

3.3 Les 5 Pièges Classiques à Éviter le Jour de l'Épreuve Pratique

1. **L'absence du Commit en écriture** : Écrire de superbes requêtes d'insertion sans jamais invoquer de `commit()` ou de validation transactionnelle. Les données n'apparaîtront jamais dans la base de données lors de la vérification par le correcteur.
2. **La confusion entre Open et ExecSQL** : Tenter de charger les résultats d'une requête de sélection `SELECT` à l'aide de la méthode `ExecSQL`, ou inversement tenter d'exécuter un `INSERT` avec `Open`. Ces deux erreurs déclenchent un plantage immédiat à l'exécution.
3. **Oublier de désactiver la requête avant modification** : Écrire `SQLQuery1.SQL.Text := ...` alors que le composant est toujours actif (`Active = True`). Lazarus lèvera instantanément une erreur d'accès interdisant la modification de la commande SQL à chaud.
4. **Inversion des paramètres nommés** : Déclarer des étiquettes de paramètres dans l'instruction SQL (ex. `:NomEleve`) et utiliser un autre nom différent lors de l'appel de `ParamByName`. Cela provoque une erreur d'indexation introuvable.
5. **Négliger les conversions de types de données** : Passer des valeurs de formulaires d'édition textuelle (`TEdit.Text`) directement à des paramètres typés déclarés comme entiers sans effectuer la conversion explicite par la fonction `StrToInt`.